

COMP6000 Chop Chop

Group Report

Connor, Wood
cw668@kent.ac.uk

Will, Haysom
wah20@kent.ac.uk

Dean, Gooding
Dmag2@kent.ac.uk

Beth, Hill
Berh2@kent.ac.uk

Jack, Fermer
Jf500@kent.ac.uk

Chop Chop



School of Computing
University of Kent

Word Count: 5996

April 4, 2024

Abstract

Following a digital recipe is tricky. Struggles reading small, unclear instructions on your mobile. Outdated website navigation. Trying to use a touch screen with sticky fingers. We offer a modern solution to this.

Chop Chop is an app that makes cooking more accessible by offering hands-free, smart recipes. Our smart recipes allow users to cook without having to worry about looking through the steps of a recipe, scrolling a screen with messy hands, or starting timers. It achieves this by using a convoluted neural network (CNN), computer vision, and a full-stack application.

We showcase our agile project management strategy. Design for both front-end and back-end. Development, results, pitfalls and successes.

Contents

Abstract	1
1 Introduction	1
2 Aims	1
2.1 Objectives	1
2.2 Stretch Objectives	1
3 Ways of Working	1
3.1 User Stories	2
3.2 Meetings	2
3.3 Communication	3
3.3.1 Discord	3
3.3.2 GitLabs Wiki	3
3.3.3 GitLabs Issue Board	3
3.3.4 In-Person	3
3.4 Gantt Chart	4
3.5 Source Control	4
4 Research	4
4.1 Existing Systems	4
4.2 Object detection	4
4.2.1 Computer Vision Frameworks	5
4.2.2 Dataset management	5
4.3 Database	6
4.4 Frontend Design	6
4.4.1 Featured Recipe	6
4.4.2 Recipe cards	7
4.4.3 Colour Palette	8
4.4.4 Figma vs Zeplin	8
4.5 Frontend Development	10
4.5.1 Vue.js	10
4.5.2 CSS vs SCSS	10
5 Design	10
5.1 Frontend	10
5.1.1 Designs: Version 1	10
5.1.2 Designs: Version 2	14
5.2 Architecture	15
5.2.1 Software	15
5.2.2 Hardware	17
6 Development	17
6.1 Backend	17
6.1.1 Controller	18
6.1.2 Interpreter	18
6.1.3 Object Detection	19
6.1.4 API	19
6.2 Frontend	19

6.2.1	Page Templates and Components	20
6.3	Database	20
6.3.1	Multimedia Gallery	22
6.4	Data Collection & Training	22
6.4.1	RoboFlow	22
6.4.2	Training YOLO with CUDA	22
6.4.3	Reviewing & Improving	23
7	Testing	25
7.1	Cooking Tests	25
7.2	User Testing	25
8	Results	25
9	Future Work	26
9.1	Events Manager	26
9.2	Smart Kitchen Integration's	26
9.3	Recipe Builder with greater cognition	26
9.4	Industry Integration	26
10	Conclusion	27
	Appendices	28
	References	29

1 Introduction

In this report, we showcase our group project called *Chop Chop*. An AI-powered cooking assistant. We will be evaluating not only the software we made but also how we made it, looking into our design, development, and ways of working. This report will first outline our research and aims. We later use our aims as metrics to critique how well we achieved our goals.

We then explain how we made *Chop Chop*, and go on to discuss what we could have done with more time.

Throughout the report, we will explain both the achievements and pitfalls we have fallen into as a group and how we dealt with them.

We made this application as a team of five people, all with previous experience developing on large projects. Following agile methodologies, we used our experience and finished our development with a prototype of a full-stack application which achieves what we set out to do.

2 Aims

We prioritised the requirements of our project into objectives and stretch objectives.

2.1 Objectives

- Front-end electron application for users to display each recipe step
- Ability to search, create, view and favourite pre-written recipes
- Create a dataset to recognise ingredients for one recipe
- ML algorithm to recognise the completion of steps using image recognition with cameras
- Ability for users to manually progress recipe steps using buttons (in case the ML model fails unexpectedly)
- Complete one full recipe run-through on a complex dish

2.2 Stretch Objectives

- Voices to read recipe steps out loud.
- Run on Raspberry Pi
- Additional ability to recognise events such as pasta boiling over and display warning steps on the user's device

3 Ways of Working

To the best of our ability, we followed a scrum-style agile approach in our project management strategy.

3.1 User Stories

At the start of our project, we decided on our user stories. They were broken into fundamental statements that the user should “feel” when they use the product.

“I should be able to progress a recipe without touching the screen”

- A Camera rig to observe the kitchen’s working areas.
- Object detection system that tells us what’s in the camera’s frame.
- System to check if the objects present satisfy the recipe step.
- Way to send the correct step to a GUI.

“It should be clear to the user what to do for each step”

- Design with thought and simplicity in mind.

“UI Should be modern and easy to use”

- UI should be intuitive and follow common design protocols.

“Our interface should be accessible for a range of different disabilities”

- Design a colour palette that is easy to see and offers good levels of contrast.
- A text-to-speech system that can verbally guide the user alongside displayed text.

3.2 Meetings

During our sprints, we scheduled one team meeting and one supervisor meeting weekly. Ideally, we arranged the supervisor meeting to take place after the team meeting, allowing us to bring any discussions raised in the team meeting to the supervisor within the same week.

Throughout the week a team document was available for members to note any questions that they wanted to raise in the meetings. This functioned very well as it meant discussions were not accidentally overlooked during the meeting and it also allowed other members to see discussion points raised and prepare before the meeting.

We also made use of the GitLabs issues board and our own gantt chart to keep ourselves on schedule throughout the project. These tools allowed us to easily check what work was still outstanding for the week and compare the progress to our initial estimates when starting out. As this was a particularly large project to undertake it was crucial we kept ourselves on track.

3.3 Communication

3.3.1 Discord

From the start we had a Discord [1] server for team communication. This included separate channels for different information "streams".

We used this platform for documenting current issues and exchanging help. We also implemented a GitLabs webhook to show current merge-requests and wiki changes meaning reviews could be conducted quickly.

3.3.2 GitLabs Wiki

As a team we fully utilised the wiki feature of GitLabs. We often wrote articles documenting our processes and helpful guides to our team members on setting up the project.

The wiki we built had a huge impact on our performance as a team, it would minimise the amount of repetitive or cluttered advice on the Discord channels and collate all our information into one place. Ultimately this meant that if a member of the team forgot something or needed to learn something quickly, they would have the information available to do so.

3.3.3 GitLabs Issue Board

Another feature of GitLabs we utilised was Issue Boards [2], these give team members a visual representation of the work that needs to be completed and by whom. Our workflow for this was very simple and had four columns:

Open This issue can be claimed or is ready to be developed.

In Development This issue is already claimed and is being developed by someone.

In Review / Testing This issue is ready to be reviewed, tested, or is waiting to be merged.

Closed This issue has been completed or discarded.

Issues would be assigned to one or more people and the board would serve as a visual reminder if something needed progressing. This was very helpful in meetings where we would review the issue board.

3.3.4 In-Person

The team met daily throughout development and we could all ask questions and speak informally with each other. We discussed the project often, and in turn would solve problems before we started developing, which saved us time.

We frequently pair programmed which aided everyone in learning all aspects of the project. This gave our team a huge sense of cohesion, allowing us to work more efficiently together since we all understood the major concepts behind our project.

3.4 Gantt Chart

We created a Gantt chart (figures:19 and 20) to record the project timeline and delegate deliverables, which was broken down into two-week sprints. Once all members approved this, it was sent to our supervisor who provided insight into improvements to ensure the project worked effectively. One insight was to add our module assessments to the chart to accurately manage our university commitments and our workload.

We broke down our Gantt charts' deliverables into "Issues" on GitLab which would move across the issue board (see section 3.3.3).

3.5 Source Control

One of the key components to *Chop Chops*' success was our well-set-up and organised GitLab repository.

From our team's year-in-industry experience, we adopted a review before merging convention. This rule ensures that changes were checked by other members before being added to the main branch. This had two main advantages. One was a hugely reduced chance of buggy code getting into the main branch. Two: you gained knowledge of how the rest of the project worked outside of your assigned issues. To enforce this, we locked the main branch from being merged and required at least one approval before the merge.

We aimed to ensure the main branch was kept as clean as possible; we were mindful of what should and should not be on the repository. This included adding rules to the git ignore file when needed, pruning unnecessary files, and ensuring consistency in naming conventions for files and folders.

4 Research

4.1 Existing Systems

Whilst researching similar projects, we found *Onionbot*[3][4]. This was a similar idea to ours but focused on the internet of things approach. It used cameras attached to a raspberry pi to identify recipe steps and servo motors to adjust temperature on the hobs in case of boil-over events.

Onionbot used its CNN model's confidence as a continuous data source to identify how far through a step the user was. For instance, frying onions' "doneness" would be judged by how confident the model was that they were fried onions. Compared to our implementation which used the confidence as a threshold for discrete markers (tags) that would indicate if a step had been completed.

We took a lot of inspiration from this project. Whilst we did not take any code from *Onionbot*, its macrostructure and execution were big influences on our implementation.

4.2 Object detection

Our project's flagship feature is our custom real-time object detection system. This would be used to detect culinary objects such as food and utensils which

would dictate when a recipe step had been completed.

4.2.1 Computer Vision Frameworks

We researched several different computer vision frameworks to determine which to use.

Faster R-CNN offers a real-time object detection algorithm version of R-CNN. “One of the most accurate object detection algorithms” [5] Meaning it would be perfect at detecting small food objects in a kitchen. The drawback is that it “requires a lot of power at inference time”, which would be a problem when we ran our program on a Raspberry Pi [6].

Detectron2- Created by Facebook AI research, it claims it “provides state-of-the-art detection and segmentation algorithms” [7] and has been used in many Facebook products such as “Smart Camera for new Portal video-calling devices” [8]. It’s built using the COCO, LVIS, CityScape, and VOC20 datasets, this creates an accurate dataset with an mAP50-95 ”83.554%”. However, in many cases was outperformed by other models. [9].

YOLO – You Only Look Once, “It is a state-of-the-art, real-time object detection system which offers extreme levels of speed and accuracy.” [10], that is around “1000x faster than RCNN and 100x faster than the Fast R-CNN model” [10]. It’s also shown to have a much higher mAP50-95 ”96.7%” compared to detectron2 (Faster RCNN-101) at ”83.554%” [9].

We decided to go with YOLOv8 due to its high mAP50-95, a large number of resources online, and also a recommendation from an ex-year-in-industry colleague.

4.2.2 Dataset management

RoboFlow [11] allowed us to collaboratively create image datasets. It allowed us to upload images we had taken for training, delegate annotation jobs out to different members of the team, and then create a dataset which can be downloaded and trained locally. It was one of the few sites offering an immense data set (max 10,000 images) with an easy-to-use interface, all free. Compared to its competitor V7 Labs [12] whose free version had a max of 1000 images.

During our research process, we tried out Roboflow and used a trial of their training cloud computers. With only 100 images we confidently detected a Rubik’s cube Figure:1, being held, rotated in the hand, and placed under different backgrounds. Despite our lack of experience, the relatively small number of images required for this and the accuracy of results gave us confidence that we could scale up to entire recipes.

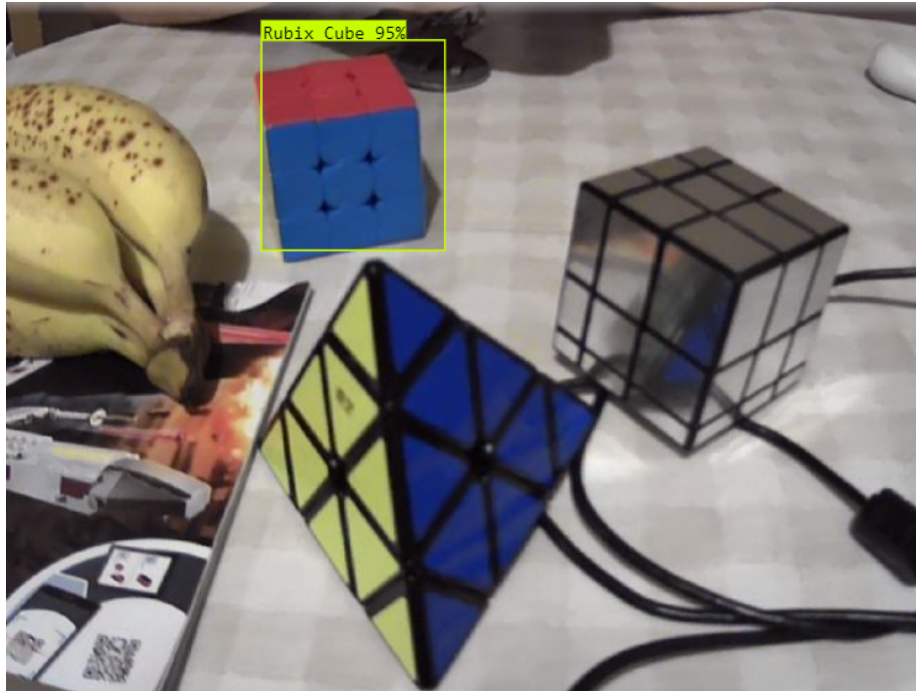


Figure 1: An initial test of making out own dataset

4.3 Database

We chose to use JSON to start with due to its quick and simple nature to set up, allowing us to quickly begin testing recipe details being displayed on the frontend.

After deciding to migrate to a relational database, we opted for SQLite over other SQL database engines. This choice was made for its straightforward implementation while still leveraging the features of SQL and not needing to start running any additional servers, which would have been required for an SQL server.

4.4 Frontend Design

Our design process began by investigating and researching other recipe applications offered in terms of layout, features, and user experience. The three main recipe applications which we focused our research on was BBC Good Foods, AllRecipes and Yummly.

4.4.1 Featured Recipe

We found that the majority of the sites showed a featured recipe which was at the forefront of the homepage, often including a background image, title, description, and a button directing users to the featured collection. We thought this was an effective way to draw users' attention to a focal point on the homepage.

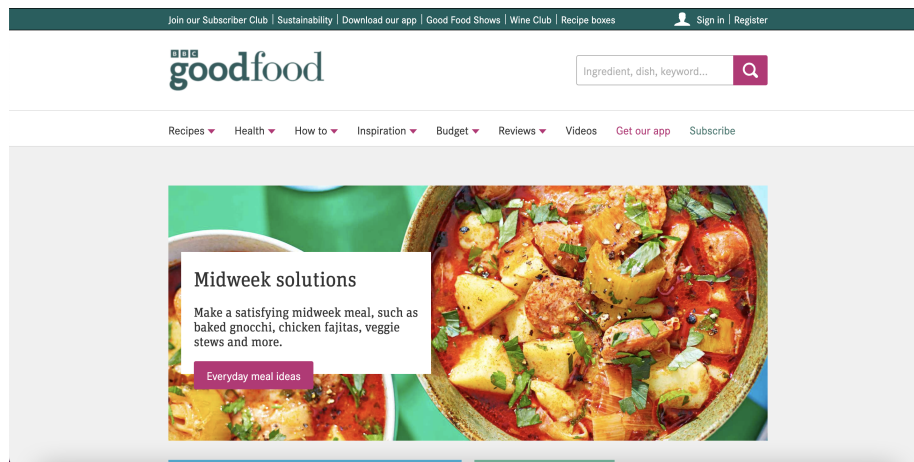


Figure 2: Homepage featured article shown on BBCGoodFoods
[13]

4.4.2 Recipe cards

Each site also used cards as a way to display each recipe. We found that this was an effective way to display each of the recipes, while only showing the necessary information needed for the user.

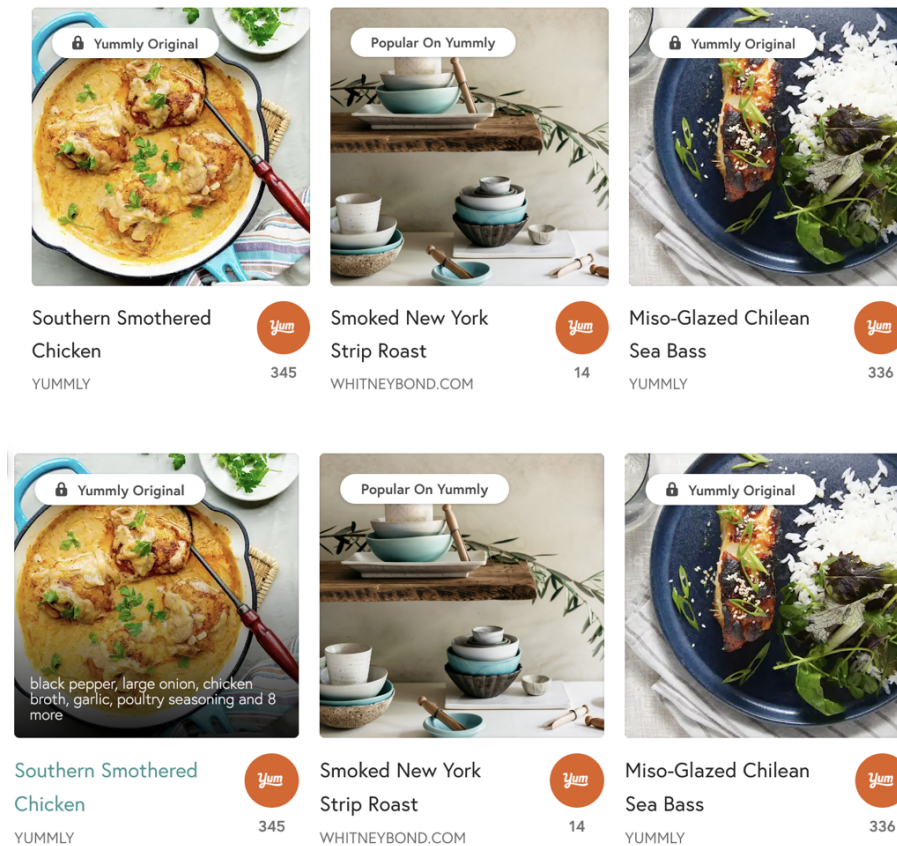


Figure 3: Recipe cards featured on Yummly
[14]

4.4.3 Colour Palette

We made sure to consider accessibility issues in our design. We found that the use of bright colours may cause pain for those who suffer with low vision or dyslexia, and may also prevent them from being able to read information from the page. This was taken into consideration with our choice of colours.

Users also need a "high contrast between the text and the background on a page" [15]. We found people "read information better on a page which incorporates dark text on a light background" [15]. We wanted to select colours for our frontend which would meet both of these accessibility criteria and found that a pale peach background with a green primary text effectively met these standards.

4.4.4 Figma vs Zeplin

Before designing prototypes of the frontend, we had to decide which software application to use. From prior experience during our placement year, we limited ourselves to either Figma or Zeplin.

We found that Zeplin is primarily a "dedicated handoff tool" [16] whereas Figma is a "collaborative design tool" [17] for teams that enables design, prototyping, and sharing within a single platform. This meant we could create designs directly using Figma while Zeplin required us using a dedicated design tool such as Sketch or Adobe XD.

As a team, we liked the use of developer tool features since we would be able to streamline the design-to-development workflow and reduce the use of repetitive, manual coding. Figma is exclusively tailored for the design process which we found to be a better experience for designing prototypes.

Below, we have inserted a table which we found within a LinkedIn article comparing the features provided on both Figma and Zeplin.

Feature	Figma	Zeplin
Real-time collaboration	✓ Allows multiple users to work simultaneously on a design file	✗ Focused on handoff, not design collaboration
Platform support	✓ Browser-based (<i>No installation required</i>) ✓ macOS/Windows (<i>Requires installation</i>)	✓ Browser-based (<i>No installation required</i>) ✓ macOS/Windows (<i>Requires installation</i>)
Prototyping	✓ Robust prototyping features	✗ No design or prototyping features
Developer handoff	✓ Built-in tools (Design specifications, asset export, CSS code)	✓ Dedicated handoff tool (Design specifications, assets, code snippets)
Jira integration	✓ Seamless project management and task tracking ✗ More focused on design collaboration, allowing users to view and comment on Figma designs within Jira	✓ Seamless project management and collaboration ✓ Zeplin's Jira integration is more focused on developer handoff, as it offers design specifications, assets, and code snippets within Jira
Learning curve	✗ Steeper (More features and interface complexity)	✓ Easier (Straightforward and intuitive interface)
Performance	✗ Can be slower (Browser-based, may lag with large projects)	✓ Generally faster (Native app, handles large projects more efficiently)
Style guide generation	✗ No automated style guide generation	✓ Automatically generates a style guide based on the design file
Cost	✓ More cost-effective Combines design and handoff capabilities	✗ Additional cost Focused on handoff, requires separate design tool

Figure 4: Table comparing features within Figma and Zeplin [16]

After considering the strengths of both applications, combined with the research which we conducted, we decided that Figma would be the best solution for us.

4.5 Frontend Development

4.5.1 Vue.js

For the frontend we considered a number of different frameworks before settling on Vue.js, Node.js (node), and electron. We discussed using Flutter for its superior mobile interfaces however, as making a mobile app became less prioritised we decided to go with vue for the following reasons:

Flexibility: Vue is flexible, we can make custom components to fit our application, we can test it on a web-browser and utilise powerful debugging tools, additionally with a bit of work we can sandbox it into a mobile application.

Familiar and easy to learn: Our team already had some experience in Vue so it would be quicker to develop. Vue's syntax and component systems are easy to learn, which worked for our team as some of us had never used a frontend framework before. This makes understanding the code much easier when it comes to reviewing merge requests or discussing approaches.

Lightweight: Vue is lightweight, it's quick to load as it is made to be run from a web-browser. Features like hot reload help us develop quickly and its' CLI tools¹ make installing packages easy.

4.5.2 CSS vs SCSS

A big decision for our application was to decide whether to use CSS or SCSS.

To begin with, CSS is the "standard stylesheet language used to define the presentation and layout of a webpage written in HTML or XML" [18]. It has a simple syntax with selectors, properties and values. On the other hand, SCSS is a "superset of CSS and it provides additional features and functionalities that are not available in regular CSS" [18] (e.g. variable, nesting, and mixins).

For our project, we decided that the use of SCSS would benefit us due to its enhanced features and flexibility. With the extra features available to us, we felt like code could be maintained and our development more efficient by the use of mixing and variables.

5 Design

5.1 Frontend

5.1.1 Designs: Version 1

The initial designs prioritised the application's functionality over user experience considerations. Inspired by our research, we designed our own components, recipe cards and recipe pages.

¹in addition to the node package manager

Featured Recipe This component was designed to include a background image of the recipe, a title and some metadata including the prep and cook time of the recipe. We felt this would still make the design eye-catching to the user, while not overwhelming the user with information.

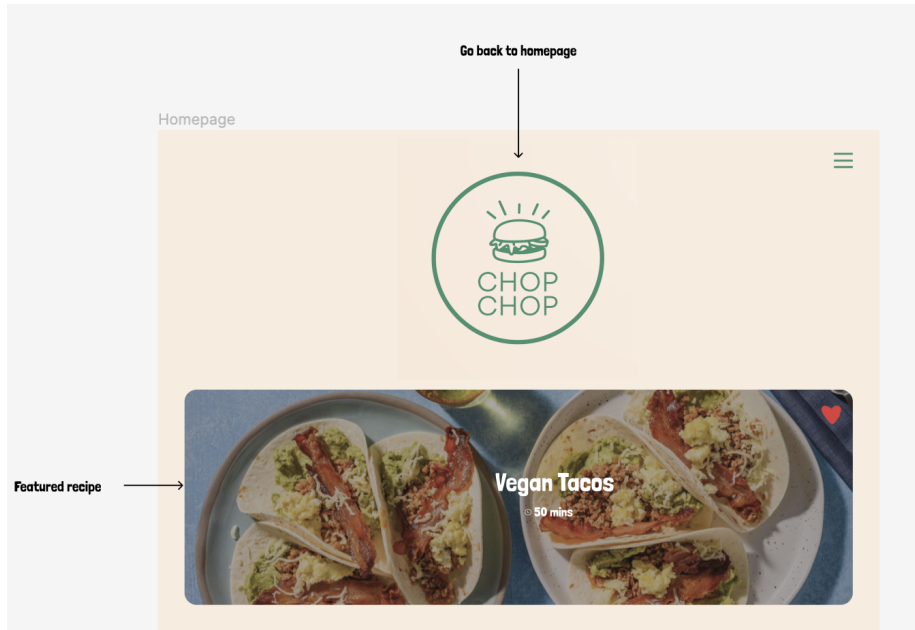


Figure 5: Featured Recipe component design

We made the functionality of this component randomly choose a recipe from the database, ensuring a dynamic and fresh experience for users with each visit to the homepage.

The component also featured a favourite button, allowing the users to save the recipe so they can build a collection of recipes which they can go back to in the future.

Recipe Cards We opted to have 2 different recipe card versions within the application: horizontal and vertical. Both versions of the cards include an image, title, like button and some metadata to display important information to the user.

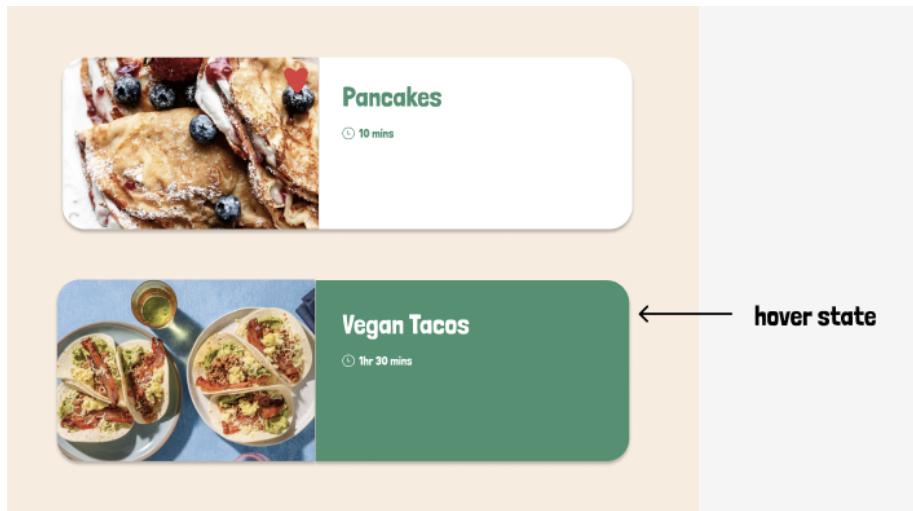


Figure 6: Horizontal recipe card design

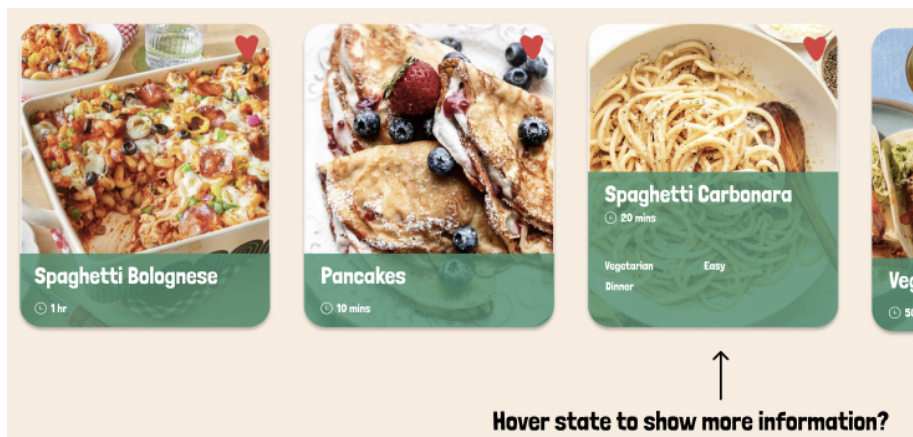


Figure 7: Vertical recipe card design

Recent Recipes We wanted a component to show recently visited recipes, enhancing the user's browsing experience and adding more simplicity to our application.

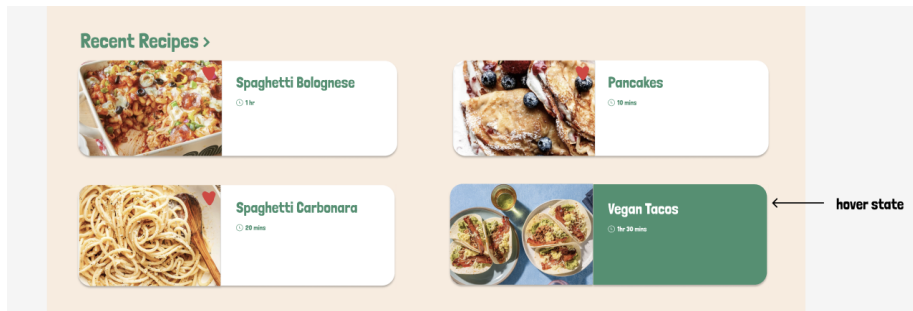


Figure 8: Recent Recipes component design

The design includes four of the recently viewed recipes, displayed using a card layout. This layout is user-friendly and visually appealing and can display each recipe in a clear and organised manner.

Liked Recipes On each recipe card, a heart-shaped icon would be shown, providing the option for users to click and add the recipe to their liked recipe collection, adding a layer of personalisation for the user.



Figure 9: Liked Recipes component design

The component features a carousel of liked recipes, which the user can scroll across, with a title which links to a liked recipe search page. These attributes ultimately allow the user to find their favourite recipes quickly.

Recipe Overview The recipe overview page showcases essential elements like ingredients and cooking steps, ensuring users understand the recipe before cooking. Divided into two sections—header and information—it offers key details and a "Start Recipe" button.

Inspired by W3Schools, the switcher component seamlessly toggles between ingredients and recipe steps.

Ingredients: Show all ingredients which are needed for the recipe, and a checkbox to allow the user to state if the ingredients have already been prepared (e.g. carrots already being chopped).

Recipe: States the steps which the user would be following.

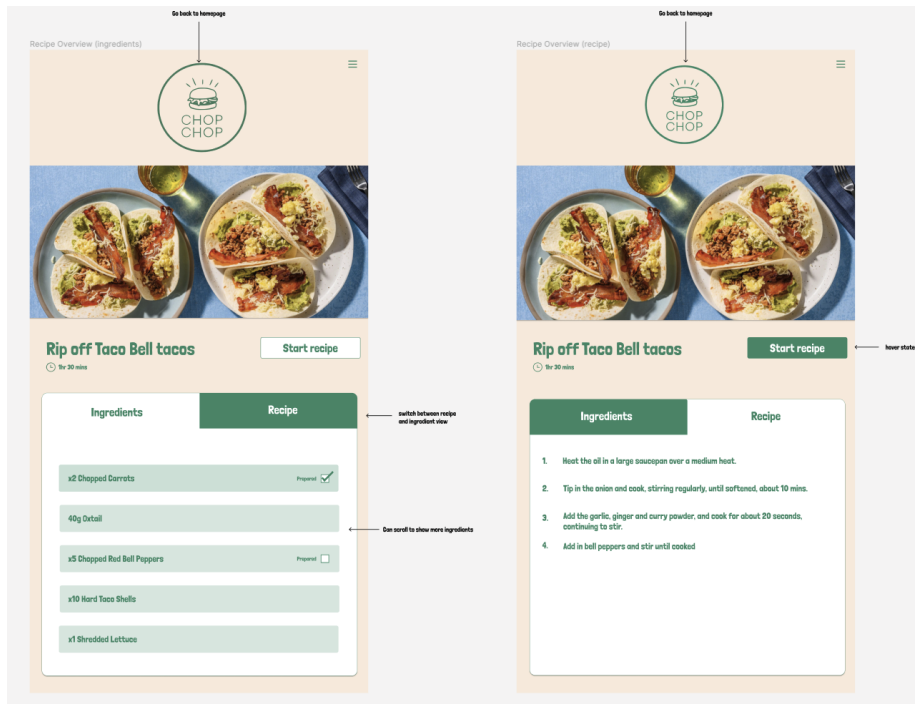


Figure 10: Recipe Overview page showing Recipe Switcher component

5.1.2 Designs: Version 2

The second iteration of our designs focused more on user experience and frontend features, elevating the overall usability and engagement of our web application.

Add Recipe Form One of the most impactful additions within this version was the "Add Recipe" form, allowing users to input their own recipes. We initially designed two versions: Design 1 had buttons on the left and forms on the right, while Design 2 utilised a 'recipe switcher' component. Opting for consistency, we chose Design 2 for its user-friendly layout, already present in other parts of the application.

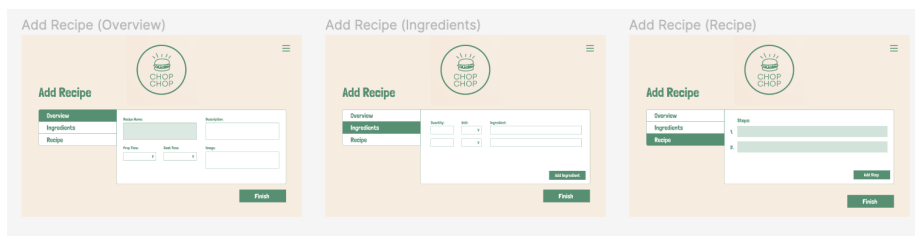


Figure 11: Add Recipe form: Design 1

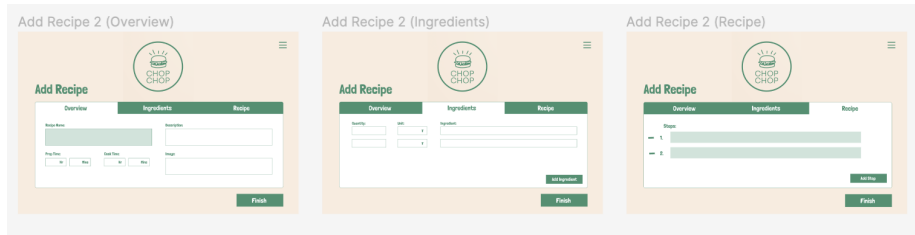


Figure 12: Add Recipe Form: Design 2

Navigation Improvements To enhance user experience and streamline navigation, we added a centralised navigation menu and back buttons throughout the application. These features aimed to provide easy access to important links and simplify user journeys, ultimately improving overall satisfaction.

Downfalls Between Version 1 and 2 of the designs, we removed some unnecessary features.

In version 2 we created a feature to add a checkbox for users to select if they had already prepared ingredients for the recipe. However, during testing, we realised that this would add unnecessary complexity to the user interface, so we decided to remove the idea.

We also removed the idea of implementing tags within recipes and recipe cards. We found that this would make the database overly complex. These tags were also not essential for the core functionality of the application.

5.2 Architecture

5.2.1 Software

Due to the size of this project, we needed to outline our goals before starting work. First, we detailed what we envisioned: a multi-platform app capable of identifying and progressing steps of a recipe. We then researched how we could execute our idea; we settled on a basic client/server approach using a client, a server, and a database. This would comprise the basis of our tech stack.

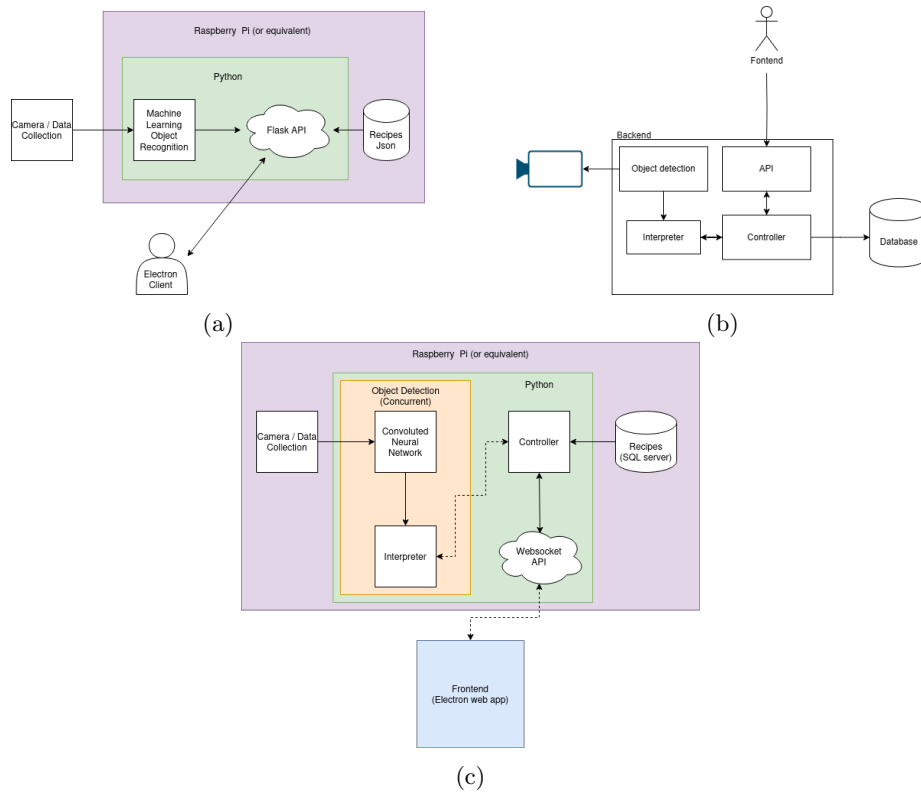


Figure 13: Our architecture drawings in chronological order

In our first diagram (13a) we identified a simplistic backend, it utilised a flask api to connect to the client and a backend without many control flows. There were cameras attached to the backend and a database which the backend could also access.

In our next drawing (13b), we detailed the backend control flow in-depth which helped us outline our python classes and files. We implemented a controller model where one central controller would keep state of the program. This was to reduce complexity by making the code modular.

After discussion, we changed our API to a websocket API to allow for multiple clients and two way communication. This also avoided the problem of long polling with a RESTful API.

We made our object detection system run concurrently so it could be interrupted or killed with ease whilst still maintaining the API. Now, the Controller and Interpreter could send messages over python threads to communicate, like when a step has been detected or what step is next.

Additionally, we changed our database from a JSON file to a relational database. This increased our application's scalability by allowing us to manage recipe data more easily rather than manipulating a plain-text JSON file.

We had lots of collaboration on the architecture, we often drew up designs on

whiteboards and discussed the advantages of approaches. This led to us being well prepared for problems and made our architecture modular by design.²

The planning of architecture contributed a lot to our process, it allowed us to develop features simultaneously knowing how they would interact ahead of time.

5.2.2 Hardware

For the hardware, we looked into running our main backend on a Raspberry Pi with web-cameras connected via USB. The Raspberry Pi would host the websocket API on our local network for any devices using the frontend to connect to. An SQL Server would also be hosted on the Raspberry Pi locally for it to access.

This way of hosting made it easier to debug as we could 'fake' messages from the frontend or backend using HTTP requests.

In reality, the Convolutional Neural Network ran inefficiently on the Raspberry Pi so we substituted it for a laptop. Our modular architecture allowed us to do this without issues.

6 Development

6.1 Backend

The backend is divided into several packages: API, Controller, Interpreter, and object detection. Additionally, we have utilities and tests, which support and are utilised within other sections. The backend is initiated from `main.py`, which launches the Controller, API, and our logging system, producing two separate log files: one called `detection` and one called `API` to help us track the status of the program. We also have `config.py`, which houses common variables used throughout the backend. This allows us to easily change any of these variables without delving into the backend. Figure 14 is a call graph which we generated using code2flow [19] to giving some extra context to how the backend is linked.

²Figure 13c, is not technically the final architecture diagram as we also added the multimedia gallery (section 6.3.1) which adds another database sending information to the frontend.

method led to long processing time between recipe steps as each step required the cameras to be reinitialised as a new thread was created. This sped up the detection time significantly, with the trade-off being that it now takes slightly longer to initialise the recipe at the start.

To help prevent false positives from progressing the step we implemented a rolling average process into the Interpreter. We experimented with the number of frames being used to calculate our rolling average and found that 7 out of a set of 10 worked well to reduce the false positives without significantly delaying progression times. This was easy to experiment with as both these variables are housed inside `config.py`.

6.1.3 Object Detection

The object detection package is designed to analyse individual frames for the presence of progression and inhibitory objects. When queried by the Interpreter, it captures the current frame using OpenCV2[20] and processes it using Supervision[21] and Ultralytics[22] libraries to extract tags for the objects in the frame. Subsequently, it determines whether progression objects are present in the list and inhibitory objects are absent, returning either true or false accordingly.

6.1.4 API

Our project's API system comprises two main components: a WebSocket server and a custom HTTP server for retrieval and uploading from the multimedia gallery. These two systems run simultaneously to achieve different parts of the API requirements.

WebSocket Server The WebSocket server facilitates real-time bidirectional communication between clients and the server, operating on asynchronous principles. It consists of two main handlers: a consumer and a producer. The handler function serves as a central component, running both consumer and producer handlers using the `asyncio` library to run simultaneously.

The consumer handler processes incoming requests from clients, parsing JSON payloads into request objects, and executing corresponding commands. These commands involve interactions with the Controller or the database.

The producer handler manages outgoing responses to clients, continuously monitoring changes in the system state. For instance, it tracks updates in recipe steps and broadcasts these updates to connected clients.

6.2 Frontend

The frontend of our application utilises HTML, SCSS, and JS, organised into page templates and components. Page templates were structured for efficient development, customisation, and component reusability. Components were individually separated into Vue.js files, enhancing scalability and code maintainability.

6.2.1 Page Templates and Components

Page Header and Navigation Menu: A prominent feature, featuring a burger button icon for toggling the navigation menu, enhanced with an animated transition using SCSS.

Recipe Cards: Implemented both horizontal and vertical recipe cards with bookmarking functionality. Clicking the bookmark icon sent a message to the API for database updates, with a flag to reload the liked recipes component upon user interaction.

Homepage The homepage serves as the central hub of the application, hosting various components like Featured Recipe, Recent Recipes, and Liked Recipes.

Recent Recipes: Utilising the horizontal recipe cards, this component displayed 4 recipes arranged using a CSS grid. During development, this component transitioned to “Implemented Recipes” to provide a designated area to separate recipes which feature the AI cooking assistant to those which do not.

Liked Recipes: Similar to the Recent Recipes component, but using a scrollable carousel to display 10 featured liked recipes. Included a title to bring the user to a listing page displaying all liked recipes.

Recipe Overview This page was used to show the user a brief overview of the recipe and contained the Recipe Switcher component. The header of this page was created using a CSS grid to display important information and a button to let the user go to the Recipe Carousel page and bookmark the recipe.

Recipe Switcher: Using the design from W3Schools, we created a JavaScript toggle function within this component that allowed the user to switch between viewing the ingredients and recipe steps.

Recipe Carousel This was the main page to show off the functionality of our project. It included a carousel and a timer component.

Carousel: A pivotal component offering clear, step-by-step instructions for recipes. Initially planned with a CodePen-inspired animation [23], technical challenges led us to retain a simpler approach, allowing steps to transition vertically.

Contained up and down buttons to allow the user to scroll through each recipe steps, as well as a back button to return the user to their current step or to the beginning of the recipe.

Timer: Shown when a recipe step requires a countdown.

6.3 Database

We originally built a JSON database, before moving onto an SQLite database later in the project.

Tables contained in the SQLite database include: Recipes, Ingredients, Steps, as well as two new ones: Voices, and Recipe Voices, to accommodate for the vocal narration of steps.

When breaking the recipes into steps we did it in a way to easily translate them into Progression and Inhibitor objects. This was important to do early as it let us know what images we would need for our dataset.

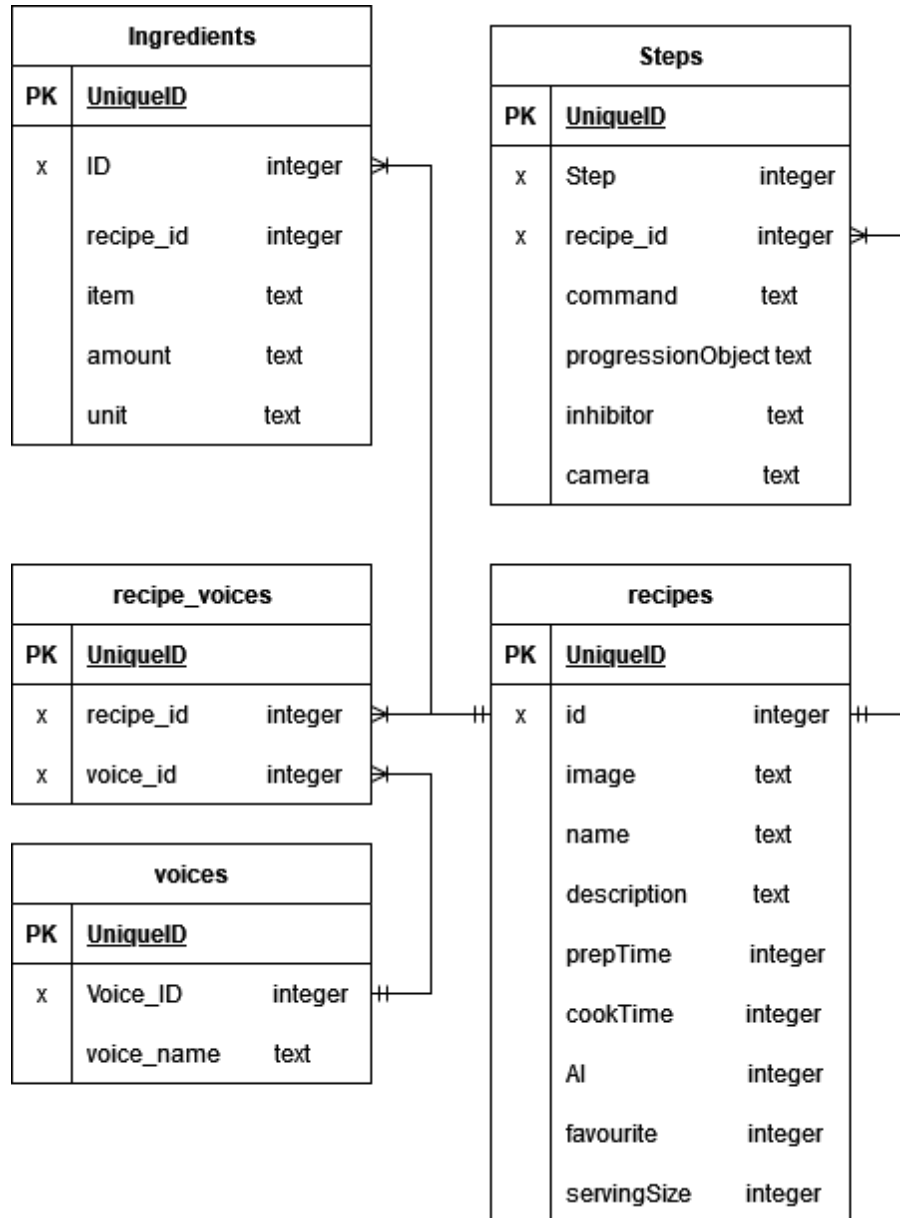


Figure 15: Entity-Relationship Diagram for our database.

6.3.1 Multimedia Gallery

The Multimedia Gallery is the file storage server storing recipes, photos and voice recordings for recipe steps. It is accessed through an API for which the URL is stored inside the SQL database.

6.4 Data Collection & Training

To begin building our object detection system, we first collected images of the objects we wanted to detect. We found it best to start with images where the object was in good lighting and well-framed before getting images of objects in different positions, with different lighting conditions and with other objects in frame.

6.4.1 RoboFlow

Once images had been gathered they were uploaded to Roboflow for annotating. We used box annotation, as it was significantly quicker to annotate and offered the same quality of detection that poly-selection did.

When all images were annotated, a new YOLOv8 dataset could be generated. Before training, Roboflow splits the images up into training images to teach, validating images to test and testing images to "provide an unbiased final evaluation" [24] of the model. Roboflow also generates extra images with different camera angles and light conditions through an augmentation process.

6.4.2 Training YOLO with CUDA

For our training we opted to use our Nivida GPU using CUDA [25]. In short "CUDA is NVIDIA's parallel computing architecture, it enables dramatic increases in computing performance, by harnessing the power of the GPU" [26]. YOLO itself uses Pytorch [27] which implements tensor cores, "similar to a multidimensional array, used to store and manipulate the inputs and outputs of a model, as well as the model's parameters. Tensors are similar to NumPy's ndarrays, except that tensors can run on GPUs to accelerate computing" [27].

By running this command we were able to train our model

```
yolo task=detect \  
  mode=train \  
  model=yolov8s.pt \  
  data={Location-Of-Dataset}/data.yaml \  
  epochs=100 \  
  imgsz=640 \  
  device=0
```

Which took in the dataset (data.yaml) from RoboFlow, and outputted weights, and results of the training.

6.4.3 Reviewing & Improving

The development of our ML model was very iterative. Our first model comprised of a small number of detectable objects but was enough to allow for the development of the rest of the project.

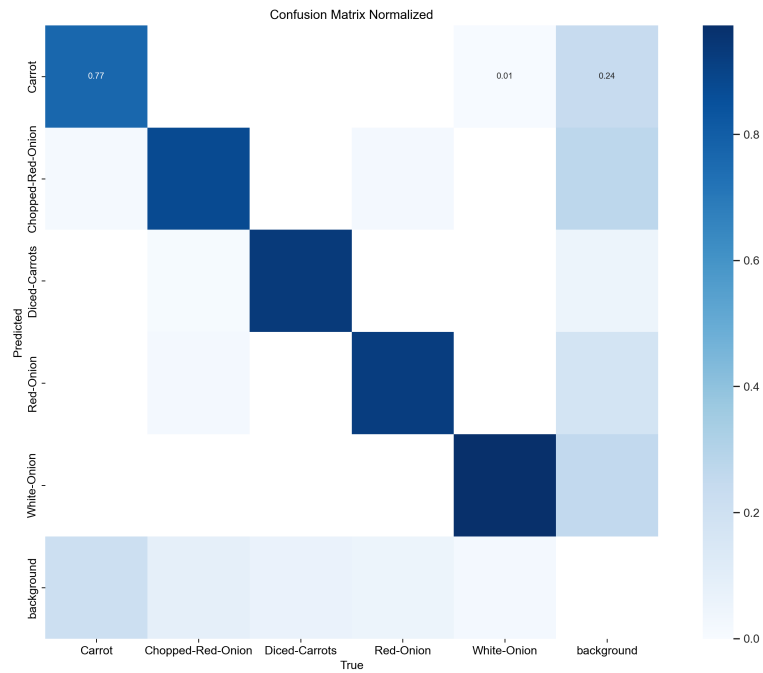


Figure 16: confusion matrix normalised of our version 1 model

As shown in the figure above, version 1 was prone to misclassifications. We found by inducing more diversity into the data set and including some null data, we could gain more accurate classifications.

7 Testing

Chop Chop, throughout development, stuck to a strong testing methodology. Each new feature had to be reviewed and tested before it was allowed to be merged in. While we shied away from writing unit tests, we always tried our best to keep the main branch bug-free.

7.1 Cooking Tests

We had the opportunity to test the full application working in our kitchen using a camera attached to a Mic stand pointing down to the prepping area, and another camera attached to the extractor fan looking at the cooking area. Both were plugged into a laptop running the *Chop Chop* app. This setup allowed us to frequently test our main recipe, the Quorn Spaghetti Bolognese.

For each test, we would record the data gathered by our program. Such as API requests, and what the object detection system saw. This would allow us to go back and analyse when the program failed. Such as when steps were not progressing, the logs helped us diagnose we were trying to compare a list of detected objects with a single object.

7.2 User Testing

Our opportunities for user testing were not huge, but we did get various friends and family to try our system. We took on feedback and decided if it was a priority to implement. This would be decided on a mix between the time complexity of the idea versus how important they and we felt it would be. One example of this would be adding serving sizes which was easy to do and important, so it was added in.

While limited, this user testing contributed towards our successful development of *Chop Chop*.

8 Results

Overall, our team worked well together, using industry standard ways of working (section 3) to produce a fully functional prototype of our application. Our software development project lead us to learning many new skills through exploration and knowledge shares, and we incorporated this learning back into *Chop Chop*.

We achieved all of our original objectives set out in section 2.1, we also achieved one of our stretch objectives from section 2.2: adding text to speech voice overs for recipe steps. However, the last two stretch objectives we did not achieve. Given more time implementation of these objectives would have been simple due to our successful modular development approach.

Our convoluted neural network is reliable with high confidence scores and low confusion on most features. We created our own dataset with 3941 images.

The project received the “Most Innovative Use of Technology” award from Holiday Extras at The University of Kent Computing Poster Fair 2024. Lecturers

and other students praised our team for our ambitious work and how smoothly our demo ran.

9 Future Work

There are a few extra features that we came up with during the development process that we did not get time to fully implement into the final project.

9.1 Events Manager

This feature would alert the user of important events in the kitchen, like the pasta boiling over ³. Effectively a second set of eyes. With most likely a visual prompt and audio alert, enhancing user awareness during cooking.

9.2 Smart Kitchen Integration's

The Event systems manager and Raspberry Pi together culminates in a future version of *Chop Chop* which would have more control over the user's kitchen. In our vision for a *Chop Chop* powered smart kitchen, our system would be able to operate kitchen devices such as the hob or oven and activate these devices when necessary. This could be used for preheating or setting timers and would improve on the warning system for the boil-over event by being able to turn down the heat of the hob on its own once the event has been recognised. This would complete the vision for *Chop Chop* to be a home cooking assistant that can take away from the stress of cooking by controlling a lot of the core processes so that the user doesn't have to.

9.3 Recipe Builder with greater cognition

Another area we would want to enhance is the process of choosing meals to cook. We propose expanding the capabilities of *Chop Chop* to include identifying a wider range of foods stored in the fridge and other areas. Currently, our image recognition can only recognise a few ingredients, but by expanding its capabilities, it could effectively determine the contents of your fridge and suggest meals based on available ingredients. This would transform *Chop Chop* into a comprehensive AI cooking assistant, guiding users from selecting ingredients to completing their meals.

9.4 Industry Integration

The final vision we had for *Chop Chop* was integration into the restaurant industry. By embedding our application into professional kitchens, we envisioned alleviating pressure experienced by chefs during peak service hours. Acting as a virtual sous chef, *Chop Chop* could monitor all ongoing meal preparations and cooking processes. This integration could involve a dedicated screen within the kitchen, offering a concise overview of current orders and their estimated completion times, significantly streamlining kitchen operations.

³Our dataset was trained and could recognise pasta boiling over but was never used

10 Conclusion

Chop Chop has been a successful project. It has achieved the objectives that were set and provided learning opportunities for the team. We researched technologies, planned meetings, and documented and developed code. The project has a fully functioning backend, frontend, and databases. We produced a ML model, with a custom dataset of 3941 images, and user-tested it thoroughly.

Our solution was novel within the existing app-space. Whilst similar projects exist such as *OnionBot* our focus on good UI design (including accessibility consideration), and User experience made *Chop Chop* a more unique and user-focused application.

There are additional opportunities to extend *Chop Chop* outlined in Future Work (section 9) which we hope to undertake in the future.

Appendices

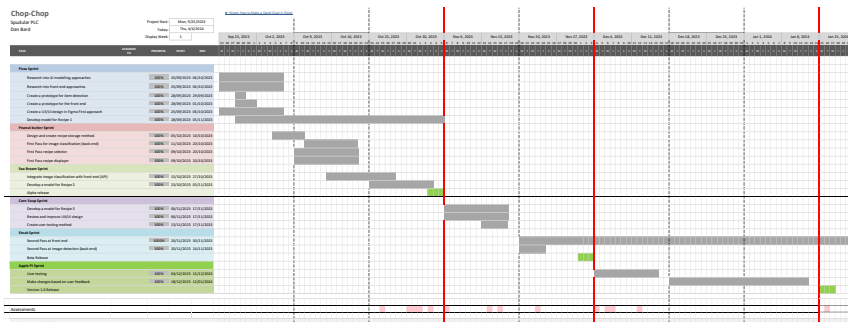


Figure 19: Gantt Chart

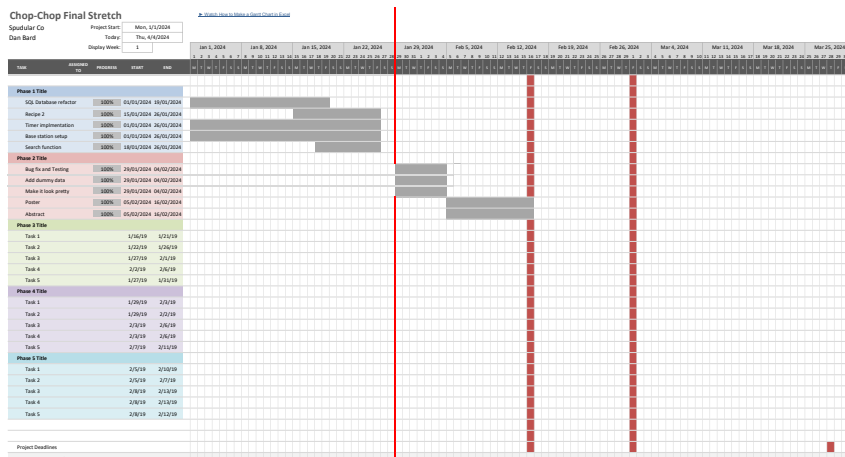


Figure 20: Gantt Chart Final Stretch

References

- [1] D. Inc, “Discord website.” <https://discord.com/>, 2024.
- [2] C.-C. Team, “Chop-chop-issue-board.” <https://git.cs.kent.ac.uk/dab46/comp6000-chop-chop/-/boards>, 2024.
- [3] B. Cobley and D. Boyle, “Onionbot: A system for collaborative computational cooking,” 2020.
- [4] B. Cobley, “Onionbot.” <https://github.com/onionbot>, 2023.
- [5] RoboFlow, “What is faster r-cnn?.” <https://roboflow.com/model/faster-r-cnn>, 2023.
- [6] EdjeElectronics, “Faster rcnn on raspberry pi - github.” <https://github.com/EdjeElectronics/TensorFlow-Object-Detection-on-the-Raspberry-Pi/issues/57>, 2019.
- [7] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2.” <https://github.com/facebookresearch/detectron2>, 2019.
- [8] Meta, “Meta smart camera portal with detectron2.” <https://ai.meta.com/blog/smart-camera-portal-advances/>, 2019.
- [9] M. Butt, N. Glas, J. Monsuur, R. Stoop, and A. de Keijzer, “Application of yolov8 and detectron2 for bullet hole detection and score calculation from shooting cards,” *AI*, vol. 5, no. 1, pp. 72–90, 2024.
- [10] P. Rajeshwari, P. Abhishek, P. Srikanth, and T. Vinod, “Object detection: an overview,” *Int. J. Trend Sci. Res. Dev.(IJTSRD)*, vol. 3, no. 1, pp. 1663–1665, 2019.
- [11] B. Dwyer, J. Nelson, T. Hansen, *et al.*, “Roboflow 1.0.” <https://roboflow.com>, 2024.
- [12] V. LTD, “V7 labs pricing.” <https://www.v7labs.com/pricing>, 2024.
- [13] I. M. C. Ltd, “Bbc good food.” <https://www.bbcgoodfood.com/>, Year Published.
- [14] Yummly, “Yummly.” <https://www.yummly.co.uk/>, 2024.
- [15] S. L. H. Jim Allan, Andrew Kirkpatrick, “Accessibility requirements for people with low vision.” <https://www.w3.org/TR/low-vision-needs/>, 2016.
- [16] E. Payamps, “Figma vs. zeplin for developer handoff: A comprehensive comparison.” <https://www.linkedin.com/pulse/figma-vs-zeplin-developer-handoff-comprehensive-edgar-payamps>, 2023.
- [17] Figma, “Figma design.” <https://www.figma.com/design/>, n.d.
- [18] geeksforgeeks, “Difference between css and scss.” <https://www.geeksforgeeks.org/what-is-the-difference-between-css-and-scss/>, 2024.
- [19] scottrogowski, “code2flow.” <https://github.com/scottrogowski/code2flow?tab=readme-ov-file>, 2023.

- [20] opencv, “opencv-python.” <https://github.com/opencv/opencv-python>, 2024.
- [21] supervision, “supervision.” <https://github.com/roboflow/supervision>, 2024.
- [22] ultralytics, “ultralytics.” <https://github.com/ultralytics/ultralytics>, 2024.
- [23] Aija, “Css vertical carousel animation.” <https://codepen.io/aija/pen/xvXWoK>, n.d.
- [24] Kili, “Training, validation and test sets: How to split machine learning data.” <https://kili-technology.com/training-data/training-validation-and-test-sets-how-to-split-machine-learning-data>, 2018.
- [25] C. CUDA, “What is cuda?”
- [26] J. Ghorpade, J. Parande, M. Kulkarni, and A. Bawaskar, “Gpgpu processing in cuda architecture,” *arXiv preprint arXiv:1202.4347*, 2012.
- [27] A. Name, “nvidia pytorch.” <https://www.nvidia.com/en-us/glossary/pytorch/>, 2024.